

The Solo Dev Survival Guide

Agile, Relational Databases, Backend Architecture, MVC, CI/CD, DevOps, and Testing —
for the One-Person Dev Team

Compiled for a Solo Developer

2026

Contents

Introduction: Welcome to the One-Person Dev Team	3
1. Agile	4
1.1 Key Principles and Values	4
1.2 Agile Frameworks: Scrum vs Kanban	4
1.3 Roles and Ceremonies Adapted for Solo Developers	5
1.4 Practical Tips to Apply Agile as a One-Person Team	5
2. Relational Databases	7
2.1 Fundamentals of Relational Databases	7
2.2 SQL Basics and Advanced Queries	8
2.3 Database Design and Normalization	9
2.4 Working with Relational Databases in Backend Projects	10
3. Backend Concepts	12
3.1 Core Backend Architecture and Components	12
3.2 RESTful APIs and Best Practices	12
3.3 Authentication, Authorization, and Security Essentials	13
3.4 Error Handling, Logging, and Performance Considerations	15
4. Layered MVC Architecture	17
4.1 Explanation of the Model-View-Controller Pattern	17
4.2 Structuring Backend Projects Using Layered MVC	17
4.3 Benefits of Separation of Concerns	20
5. CI/CD (Continuous Integration and Continuous Deployment)	21
5.1 Importance and Benefits of CI/CD	21
5.2 Setting Up CI/CD Pipelines for Backend Projects	21
5.3 Tools and Workflows Suitable for Solo Developers	22
6. DevOps Basics	24
6.1 Overview of DevOps Culture and Practices	24
6.2 Infrastructure as Code, Monitoring, and Automation Basics	24

6.3 Managing Deployments, Scaling, and Environments Solo	26
7. Unit Testing and QA Testing	28
7.1 Importance of Testing in Software Quality	28
7.2 Writing Effective Unit Tests for Backend Code	28
7.3 Automated Testing and Integration Testing	30
7.4 Strategies for Quality Assurance When Working Alone	31
Putting It All Together: A Solo Dev Weekly Rhythm	33
Final Notes	33

Introduction: Welcome to the One-Person Dev Team

If you're building something like a full-stack MERN application by yourself, you are simultaneously the Product Owner, Scrum Master, Developer, DBA, DevOps Engineer, and QA Tester. That sounds overwhelming — but it's also a superpower. You don't need meetings to align with “the team,” because the team is you. This guide strips down every major software engineering discipline to its essential, practical core so you can run a professional-grade workflow without the overhead designed for 10-person teams.

Each section below is self-contained, beginner-friendly, but detailed enough to actually use on a real project.

1. Agile

1.1 Key Principles and Values

Agile started as a reaction against rigid, document-heavy “waterfall” development. The **Agile Manifesto** (2001) boils down to four value statements:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a rigid plan

For a solo developer, “customer collaboration” might mean collaboration with your future self, your professor, your client, or your end users — anyone whose feedback shapes the product.

The 12 principles behind the manifesto distill into a few that matter most for solo work:

- **Deliver working software frequently** (weekly or bi-weekly, not “all at once at the end”).
- **Working software is the primary measure of progress** — not a Gantt chart, not a spec document.
- **Simplicity** — maximizing the amount of work *not* done is an art.
- **Continuous attention to technical excellence** — refactor as you go, don’t let debt pile up.
- **Reflect and adjust regularly** — even solo, take time to review what’s working.

1.2 Agile Frameworks: Scrum vs Kanban

Scrum

Scrum organizes work into fixed-length **sprints** (commonly 1-2 weeks). At the start of a sprint you commit to a set of tasks (the **Sprint Backlog**), and at the end you review what got done.

Core artifacts:

- **Product Backlog** — the master list of everything that could be built, prioritized.
- **Sprint Backlog** — the subset of the backlog you’re tackling this sprint.
- **Increment** — the working, shippable result of the sprint.

Core ceremonies (traditionally):

- Sprint Planning
- Daily Standup
- Sprint Review
- Sprint Retrospective

Kanban

Kanban is a continuous-flow system built around a visual board with columns like:

Backlog | To Do | In Progress | In Review | Done

Instead of fixed sprints, work flows continuously, and you limit **Work In Progress (WIP)** — e.g., never have more than 2 cards “In Progress” at once. This is often a *better fit for solo developers* because it removes the overhead of sprint ceremonies while still giving visual structure.

Which Should a Solo Dev Use?

Aspect	Scrum	Kanban	Solo Recommendation
Time-boxing	Fixed sprints	Continuous	Kanban, or “soft sprints” (loose weekly goals)
Ceremonies	Many	Minimal	Kanban — lighter overhead
Best for	Teams needing rhythm/accountability	Individuals, ongoing maintenance	Kanban with light Scrum habits

A common hybrid — “**Scrumban**” — is ideal: use a Kanban board for visual flow, but borrow Scrum’s weekly planning and retrospective habits for structure.

1.3 Roles and Ceremonies Adapted for Solo Developers

In a real team, Scrum defines three roles: **Product Owner** (decides *what* to build), **Scrum Master** (facilitates process), and **Development Team** (builds it). As a solo dev, you wear all three hats — but consciously *switching hats* at different times keeps you focused and prevents you from coding without direction or planning forever without shipping.

Ceremony	Team Version	Solo Adaptation
Sprint Planning	Team agrees on sprint goals	Spend 15-30 min every Monday (or sprint start) writing 3-7 user stories/tasks for the week
Daily Standup	“What I did / will do / blockers”	A 2-minute journal entry each morning — even one sentence: “Today: fix slug index bug, write 2 tests”
Sprint Review	Demo to stakeholders	Record a 2-minute screen capture or write a changelog entry showing what now works
Sprint Retrospective	“What went well / poorly / change”	Friday 10-min reflection: 1 thing that worked, 1 thing that didn’t, 1 change for next week
Backlog Refinement	Team re-prioritizes backlog	Every week, reorder your task board, delete stale/irrelevant tasks

1.4 Practical Tips to Apply Agile as a One-Person Team

1. **Use a Kanban board.** Free tools: GitHub Projects (built into your repo — great if you’re already using GitHub), Trello, Notion. Columns: **Backlog**, **This Week**, **In Progress**, **Testing**, **Done**.

2. **Write user stories, even for yourself.** Format: *“As a [user type], I want to [do something], so that [benefit].”* Example: *“As a visitor, I want to filter articles by category, so that I can find relevant content faster.”* This forces you to think from the user’s perspective, not just “add a filter dropdown.”
3. **Time-box your sprints.** A 1-week sprint is ideal for solo work — long enough to get something meaningful done, short enough that you can re-plan quickly if priorities shift.
4. **Limit your WIP.** Don’t have 6 half-finished features. Cap “In Progress” at 1-2 items. Finish before starting new things — this is the single highest-leverage Agile habit for a solo dev.
5. **Define “Done.”** For each task, write a one-line “Definition of Done” — e.g., “Done = endpoint works, has a test, and is documented in the README.” Without this, “done” silently becomes “it compiles.”
6. **Keep a changelog.** A simple `CHANGELOG.md` updated every sprint review gives you a running history of progress — useful for portfolios, resumes, and your own sanity when debugging “when did this break?”
7. **Don’t skip retrospectives just because you’re alone.** Self-reflection is where you catch patterns — “I keep underestimating database tasks,” “I get distracted by polishing UI before the backend works.”
8. **Avoid over-planning.** Agile rejects big upfront design. Plan just enough to start, then adapt. A backlog with 200 perfectly-groomed tickets you’ll never get to is waterfall in disguise.

2. Relational Databases

Even if your current project (feria-webprog) uses MongoDB, understanding relational database theory is essential — it underlies how *all* structured data should be modeled, and many real-world backend roles require SQL.

2.1 Fundamentals of Relational Databases

A **relational database** organizes data into **tables** (also called *relations*), where:

- Each **row** (record/tuple) represents one entity instance.
- Each **column** (attribute/field) represents a property of that entity.
- Each table has a **primary key (PK)** — a unique identifier for each row.
- Relationships between tables are expressed via **foreign keys (FK)** — a column in one table that references the primary key of another.

Example: a blog database might have tables `users`, `articles`, and `comments`:

`users`

```
+-----+-----+-----+
| id | username | email          |
+-----+-----+-----+
| 1  | ian      | ian@example.com |
+-----+-----+-----+
```

`articles`

```
+-----+-----+-----+-----+
| id | title      | slug      | user_id |
+-----+-----+-----+-----+
| 1  | Hello World| hello-w  | 1       |
+-----+-----+-----+-----+
```

`comments`

```
+-----+-----+-----+-----+
| id | article_id | user_id  | body    |
+-----+-----+-----+-----+
| 1  | 1          | 1        | Nice!   |
+-----+-----+-----+-----+
```

Here, `articles.user_id` is a foreign key referencing `users.id`, and `comments.article_id` / `comments.user_id` reference `articles.id` and `users.id` respectively.

Relationship Types

- **One-to-Many (1:N)** — One user can write many articles. Most common relationship.
- **Many-to-Many (M:N)** — Articles can have many tags, and tags can belong to many articles. Requires a **junction table** (e.g., `article_tags` with `article_id` and `tag_id`).
- **One-to-One (1:1)** — One user has one profile. Less common; often just merged into one table unless there's a reason to separate (e.g., optional/sensitive data).

2.2 SQL Basics and Advanced Queries

Data Definition Language (DDL) — defining structure

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE articles (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    slug VARCHAR(255) UNIQUE NOT NULL,  
    body TEXT,  
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,  
    created_at TIMESTAMP DEFAULT NOW()  
);
```

Data Manipulation Language (DML) — CRUD basics

```
-- Create  
INSERT INTO users (username, email) VALUES ('ian', 'ian@example.com');  
  
-- Read  
SELECT * FROM users WHERE username = 'ian';  
  
-- Update  
UPDATE users SET email = 'newemail@example.com' WHERE id = 1;  
  
-- Delete  
DELETE FROM users WHERE id = 1;
```

Advanced Queries

JOINS — combining data from multiple tables:

```
-- Get all articles with their author's username  
SELECT articles.title, users.username  
FROM articles  
INNER JOIN users ON articles.user_id = users.id;  
  
-- LEFT JOIN: include users even if they have no articles  
SELECT users.username, articles.title  
FROM users  
LEFT JOIN articles ON articles.user_id = users.id;
```

Aggregation and Grouping:

```
-- Count articles per user
```



```
SELECT users.username, COUNT(articles.id) AS article_count
FROM users
LEFT JOIN articles ON articles.user_id = users.id
GROUP BY users.username
ORDER BY article_count DESC;
```

```
-- Filter groups with HAVING
SELECT user_id, COUNT(*) AS total
FROM articles
GROUP BY user_id
HAVING COUNT(*) > 5;
```

Subqueries:

```
-- Users who have never posted an article
SELECT username FROM users
WHERE id NOT IN (SELECT DISTINCT user_id FROM articles);
```

Indexes (critical for performance — and directly relevant to your slug-index bug):

```
-- Speeds up lookups by slug, and enforces uniqueness
CREATE UNIQUE INDEX idx_articles_slug ON articles(slug);

-- Drop an outdated/incorrect index
DROP INDEX idx_articles_slug;
```

Transactions — ensuring multiple operations succeed or fail together:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- If anything fails: ROLLBACK;
```

2.3 Database Design and Normalization

Normalization is the process of organizing tables to minimize redundancy and avoid data anomalies.

- **1NF (First Normal Form):** Each column holds atomic (indivisible) values; no repeating groups.
 - Bad: tags: "tech, news, opinion" in one column.
 - Good: a separate tags table with one row per tag, linked via a junction table.
- **2NF:** Must be in 1NF, and every non-key column depends on the *whole* primary key (relevant for composite keys).
- **3NF:** Must be in 2NF, and no non-key column depends on another non-key column (no “transitive dependencies”).
 - Bad: storing both user_id and username in the articles table — if the username changes, you’d need to update every article.

- Good: store only `user_id`, and join to `users` to get the username.

When to Denormalize

Normalization isn't always "more is better." Sometimes you intentionally **denormalize** (duplicate data) for performance — e.g., storing a `comment_count` directly on the `articles` table so you don't have to `COUNT()` every time you list articles. This is a deliberate trade-off between read speed and write complexity, and it's exactly the kind of decision a solo dev should document with a comment in the schema.

Entity-Relationship Diagrams (ERDs)

Before writing any `CREATE TABLE` statements, sketch your entities and relationships. Even a rough diagram (boxes for tables, lines for relationships, with cardinality like `1 --- N`) prevents painful schema migrations later. Free tools: dbdiagram.io, draw.io, or even pen and paper.

2.4 Working with Relational Databases in Backend Projects

Even though `feria-webprog` uses MongoDB, these patterns map closely:

Relational Concept	MongoDB/Mongoose Equivalent
Table	Collection
Row	Document
Primary Key	<code>_id</code> (auto-generated <code>ObjectId</code>)
Foreign Key	<code>ref</code> field + <code>populate()</code>
JOIN	<code>.populate('user_id')</code>
Unique Index	<code>schema.index({ slug: 1 }, { unique: true })</code>
Normalization	Referencing (separate collections)
Denormalization	Embedding (nested documents)

Example: Mongoose schema with a relationship (the relational mindset applied to Mongo):

```
const articleSchema = new mongoose.Schema({
  title: { type: String, required: true },
  slug: { type: String, required: true, unique: true },
  body: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true });

// Fetch articles with author info populated (like a SQL JOIN)
const articles = await Article.find().populate('author', 'username email');
```

If you ever do use a relational DB (PostgreSQL/MySQL) in a Node backend, common tools are:

- **Sequelize** or **Prisma** (ORMs — write JS/TS, get SQL)
- **pg** / **mysql2** (raw query drivers)

Example with Prisma:

```

// schema.prisma
model User {
  id      Int      @id @default(autoincrement())
  username String   @unique
  articles Article[]
}

model Article {
  id      Int      @id @default(autoincrement())
  title   String
  slug    String   @unique
  authorId Int
  author  User     @relation(fields: [authorId], references: [id])
}

// Query
const articles = await prisma.article.findMany({
  include: { author: true },
});

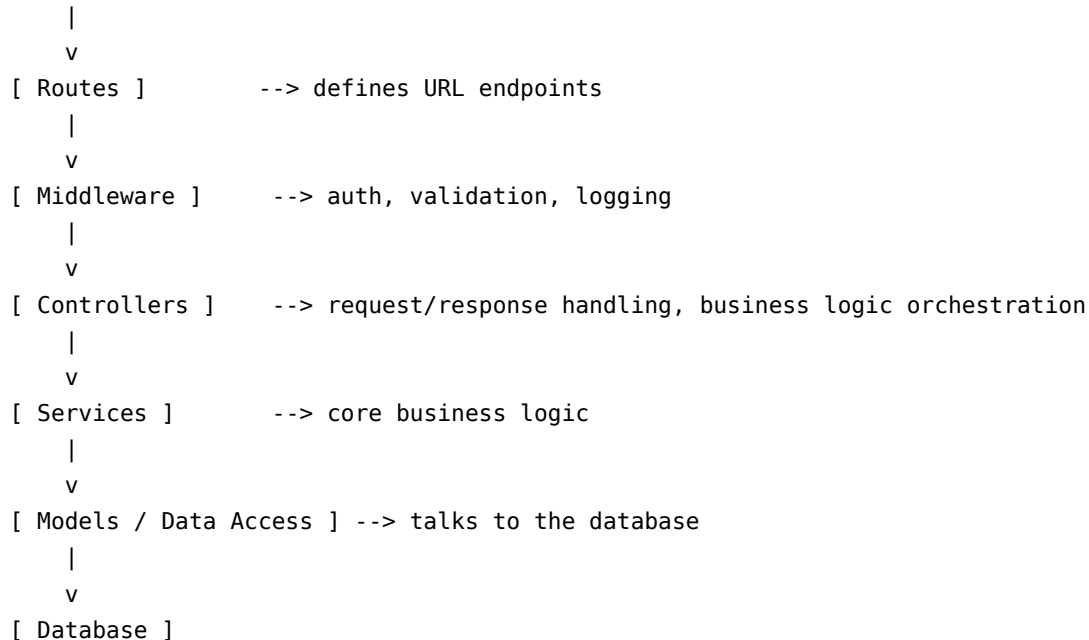
```

3. Backend Concepts

3.1 Core Backend Architecture and Components

A typical backend application is composed of several layers working together:

Client (Browser/App)



Key Components

- **Server** — the running process (e.g., an Express app listening on a port).
- **Routing** — maps HTTP methods + URL paths to handler functions (GET /api/articles).
- **Middleware** — functions that run *between* the request arriving and the response being sent (auth checks, body parsing, logging, error handling).
- **Controllers** — receive the request, call business logic, and shape the response.
- **Services / Business Logic** — the actual “rules” of your app (e.g., “a slug must be unique,” “a user can only edit their own articles”).
- **Models** — define data shape and talk to the database (Mongoose models, Sequelize models, etc.).
- **Database** — persistent storage (MongoDB, PostgreSQL, etc.)

3.2 RESTful APIs and Best Practices

REST (Representational State Transfer) is an architectural style for APIs based on resources and standard HTTP verbs.

Resource-Based URLs

GET	/api/articles	-> list all articles
GET	/api/articles/:id	-> get one article
POST	/api/articles	-> create an article
PUT	/api/articles/:id	-> replace an article

PATCH /api/articles/:id -> partially update an article
DELETE /api/articles/:id -> delete an article

Best Practices

1. Use **nouns, not verbs, in URLs**. /api/articles, not /api/getArticles.
2. Use **plural nouns** for collections: /articles, not /article.
3. **Nest resources logically** for relationships: /api/articles/:id/comments.
4. Use proper **HTTP status codes**:

Code	Meaning	Use Case
200	OK	Successful GET/PUT/PATCH
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Validation failed
401	Unauthorized	Missing/invalid auth
403	Forbidden	Authenticated but not allowed
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate (e.g., slug already exists)
422	Unprocessable Entity	Semantically invalid data
500	Internal Server Error	Unhandled server error

5. **Consistent response shape**. Pick a format and stick to it:

```
// Success
{ "success": true, "data": { ... } }

// Error
{ "success": false, "error": { "message": "Slug already exists", "code": "DUPLICATE_SLUG" } }
```

6. **Support pagination, filtering, sorting** for list endpoints:

GET /api/articles?page=2&limit=10&sort=-createdAt&category=tech

7. **Version your API** if you expect breaking changes: /api/v1/articles.
8. **Idempotency** — PUT, DELETE, and GET should be safe to repeat without side effects (calling DELETE /articles/5 twice shouldn't error the second time in a well-designed API — return 404 or 204 gracefully).

3.3 Authentication, Authorization, and Security Essentials

Authentication vs Authorization

- **Authentication** = “Who are you?” (login, verifying identity)
- **Authorization** = “What are you allowed to do?” (permissions, roles)

Common Authentication Approaches

JWT (JSON Web Tokens) — most common for REST APIs:

```

const jwt = require('jsonwebtoken');

// On login
const token = jwt.sign(
  { userId: user._id, role: user.role },
  process.env.JWT_SECRET,
  { expiresIn: '1h' }
);

// Middleware to verify
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

```

Authorization middleware (role-based):

```

function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  };
}

// Usage
router.delete('/articles/:id', authenticate, requireRole('admin'), deleteArticle);

```

Security Essentials Checklist

1. Never store plain-text passwords. Use bcrypt:

```

const bcrypt = require('bcrypt');

const hashedPassword = await bcrypt.hash(password, 10);
const isMatch = await bcrypt.compare(inputPassword, hashedPassword);

```

2. Use environment variables for secrets (.env + dotenv), never commit .env to git.
3. Validate and sanitize all input — use express-validator or Joi/zod.
4. Set security headers with helmet:

```
const helmet = require('helmet');
app.use(helmet());
```

5. **Rate limiting** to prevent brute-force attacks:

```
const ratelimit = require('express-rate-limit');
app.use('/api/auth', ratelimit({ windowMs: 15 * 60 * 1000, max: 10 }));
```

6. **CORS configuration** — only allow trusted origins:

```
const cors = require('cors');
app.use(cors({ origin: 'https://yourfrontend.com' }));
```

7. **Prevent injection attacks** — Mongoose/Prisma parameterize queries automatically, but never build queries via string concatenation with raw input.

8. **HTTPS everywhere** in production (handled by your hosting platform — Render, Vercel, etc.)

3.4 Error Handling, Logging, and Performance Considerations

Centralized Error Handling

```
// errorHandler.js – must be the LAST middleware registered
function errorHandler(err, req, res, next) {
  console.error(err.stack);

  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    error: {
      message: err.message || 'Internal Server Error',
      ...(process.env.NODE_ENV === 'development' && { stack: err.stack }),
    },
  });
}
```

```
module.exports = errorHandler;
```

```
// app.js
app.use('/api', routes);
app.use(errorHandler); // registered after all routes
```

Custom Error Classes

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}
```

```
// Usage in a controller
```

```
if (existingArticle) {  
  throw new AppError('An article with this slug already exists', 409);  
}
```

Logging

- **Development:** morgan for HTTP request logs.
- **Production:** structured logging with winston or pino, with log levels (error, warn, info, debug).

```
const morgan = require('morgan');  
app.use(morgan('dev'));  
  
const winston = require('winston');  
const logger = winston.createLogger({  
  level: 'info',  
  format: winston.format.json(),  
  transports: [  
    new winston.transports.File({ filename: 'error.log', level: 'error' }),  
    new winston.transports.Console(),  
  ],  
});
```

Performance Considerations

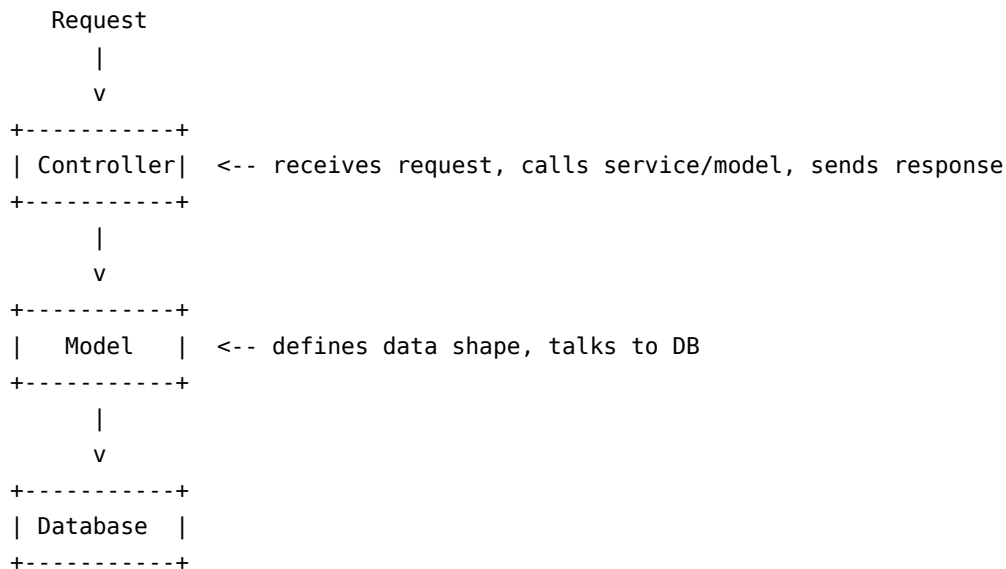
- **Database indexing** on frequently queried fields (you're already dealing with this for slugs!)
- **Pagination** — never return unbounded result sets.
- **Caching** — Redis for frequently accessed, rarely changing data.
- **Avoid N+1 queries** — use `.populate()` / `JOIN` / `include` instead of looping and querying inside the loop.
- **Async/await properly** — don't block the event loop with synchronous heavy computation; offload to worker threads or queues if needed.
- **Connection pooling** — let your DB driver manage connections (Mongoose and Prisma do this by default).
- **Compression** — `app.use(compression())` for response payloads.

4. Layered MVC Architecture

4.1 Explanation of the Model-View-Controller Pattern

MVC separates an application into three interconnected components:

- **Model** — represents data and business rules. In a backend API, this is your database schemas (Mongoose models) plus any data-access logic.
- **View** — the presentation layer. In a traditional server-rendered app, this is your templates (EJS, Pug). In a MERN app, the “View” is effectively your **React frontend**, consuming the API.
- **Controller** — receives input (HTTP requests), coordinates between Model and View, and returns a response.



(View = React frontend, communicates via JSON over HTTP)

4.2 Structuring Backend Projects Using Layered MVC

A well-organized Express/Node project (“layered MVC”, sometimes extended to MVCS — Model-View-Controller-Service):

```
feria-webprog-backend/
├─ src/
│  └─ config/
│     └─ db.js           # MongoDB connection setup
│     └─ env.js          # environment variable loading
│  └─ models/
│     └─ User.js
│     └─ Article.js
│  └─ routes/
│     └─ userRoutes.js
│     └─ articleRoutes.js
└─ controllers/
```

```

|   |   ├── userController.js
|   |   └── articleController.js
|   ├── services/
|   |   ├── userService.js
|   |   └── articleService.js
|   ├── middleware/
|   |   ├── authMiddleware.js
|   |   ├── errorHandler.js
|   |   └── validateRequest.js
|   ├── utils/
|   |   └── slugify.js
|   ├── app.js           # Express app setup, middleware registration
|   └── server.js        # entry point, starts the server
└── tests/
    ├── unit/
    └── integration/
└── .env
└── .env.example
└── package.json
└── README.md

```

Example: Each Layer in Code

Route (routes/articleRoutes.js) — defines the endpoint, delegates to controller:

```

const express = require('express');
const router = express.Router();
const articleController = require('../controllers/articleController');
const { authenticate } = require('../middleware/authMiddleware');

router.get('/', articleController.getArticles);
router.post('/', authenticate, articleController.createArticle);

module.exports = router;

```

Controller (controllers/articleController.js) — handles HTTP-specific logic, delegates business logic to service:

```

const articleService = require('../services/articleService');

exports.createArticle = async (req, res, next) => {
  try {
    const article = await articleService.createArticle(req.body, req.user.userId);
    res.status(201).json({ success: true, data: article });
  } catch (err) {
    next(err); // forwards to centralized error handler
  }
};

```

```

exports.getArticles = async (req, res, next) => {
  try {
    const { page = 1, limit = 10 } = req.query;
    const articles = await articleService.getArticles({ page, limit });
    res.json({ success: true, data: articles });
  } catch (err) {
    next(err);
  }
};

```

Service (services/articleService.js) — business logic, independent of HTTP:

```

const Article = require('../models/Article');
const AppError = require('../utils/AppError');
const slugify = require('../utils/slugify');

exports.createArticle = async (data, userId) => {
  const slug = slugify(data.title);

  const existing = await Article.findOne({ slug });
  if (existing) {
    throw new AppError('An article with this slug already exists', 409);
  }

  return Article.create({ ...data, slug, author: userId });
};

exports.getArticles = async ({ page, limit }) => {
  return Article.find()
    .populate('author', 'username')
    .sort('-createdAt')
    .skip((page - 1) * limit)
    .limit(Number(limit));
};

```

Model (models/Article.js):

```

const mongoose = require('mongoose');

const articleSchema = new mongoose.Schema({
  title: { type: String, required: true },
  slug: { type: String, required: true, unique: true },
  body: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true });

module.exports = mongoose.model('Article', articleSchema);

```

4.3 Benefits of Separation of Concerns

1. **Testability** — services can be unit-tested without spinning up an HTTP server. Controllers can be tested separately with mocked services.
2. **Maintainability** — when the slug-uniqueness bug needs fixing, you know exactly where to look: the service layer, not scattered across route handlers.
3. **Reusability** — the same `articleService.createArticle()` could be called from a REST controller, a GraphQL resolver, a CLI script, or a background job — without duplicating logic.
4. **Easier onboarding** (for future-you, or collaborators) — a predictable structure means anyone can find “where does X happen” quickly.
5. **Swappable layers** — you could swap MongoDB for PostgreSQL by rewriting the Model layer only, if your Service layer doesn’t leak database-specific details (e.g., avoid passing raw Mongoose query objects up to controllers).
6. **Reduced cognitive load** — each file does one thing. A 50-line controller is easier to reason about than a 500-line route file mixing validation, business logic, and database calls.

5. CI/CD (Continuous Integration and Continuous Deployment)

5.1 Importance and Benefits of CI/CD

- **Continuous Integration (CI)** — automatically building and testing your code every time you push changes, catching bugs before they reach production.
- **Continuous Deployment/Delivery (CD)** — automatically deploying tested code to staging or production environments.

Why It Matters, Even Solo

1. **Catch bugs early** — a failing test on push is far cheaper than a failing deploy in production.
2. **Consistency** — the pipeline runs the same checks every time, removing “works on my machine” issues.
3. **Confidence to ship often** — Agile says “deliver frequently”; CI/CD makes that safe.
4. **Free automation** — GitHub Actions gives generous free minutes for personal repos.
5. **Documentation as code** — your pipeline config *is* a description of your build/test/deploy process, useful for future-you and anyone reviewing your project (including recruiters/professors).

5.2 Setting Up CI/CD Pipelines for Backend Projects

Basic CI Pipeline (GitHub Actions)

.github/workflows/ci.yml:

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci
```

```

- name: Run linter
  run: npm run lint

- name: Run tests
  run: npm test
  env:
    MONGO_URI: ${ secrets.MONGO_URI_TEST }
    JWT_SECRET: ${ secrets.JWT_SECRET }

```

Adding CD: Auto-Deploy on Merge to Main

Most modern hosting platforms (Render, Railway, Vercel, Netlify) have **native GitHub integration** — they auto-deploy on push to a branch without needing custom YAML. This is the simplest “CD” setup for a solo dev:

1. Connect your GitHub repo to Render/Railway (backend) and Vercel/Netlify (frontend).
2. Configure: deploy from `main` branch only.
3. Set environment variables in the platform’s dashboard (not in code).
4. Done — every merge to `main` that passes CI gets deployed automatically.

If you want an explicit deploy step in GitHub Actions instead (e.g., deploying via SSH or Docker):

```

deploy:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Deploy to Render
      run: curl -X POST "${ secrets.RENDER_DEPLOY_HOOK_URL }"

```

Monorepo Considerations (Backend + Frontend)

If `feria-webprog` has both `/backend` and `/frontend` in one repo, use path filters so changes to one don’t trigger unnecessary builds of the other:

```

on:
  push:
    paths:
      - 'backend/**'

```

5.3 Tools and Workflows Suitable for Solo Developers

Tool	Purpose	Why It’s Solo-Friendly
GitHub Actions	CI/CD pipelines	Free tier, integrated with GitHub, huge community of example workflows
Render / Railway	Backend hosting	Free/cheap tiers, auto-deploy from GitHub, built-in env var management

Tool	Purpose	Why It's Solo-Friendly
Vercel / Netlify	Frontend hosting	Instant previews per PR, zero-config for React/Vite
MongoDB Atlas	Managed database	No DB server to maintain (you're already using this)
Husky + lint-staged	Pre-commit hooks	Catch issues <i>before</i> they're even pushed
Dependabot	Dependency updates	Automated PRs for security patches

Pre-commit Hooks (Catch Issues Before CI Even Runs)

```

npm install --save-dev husky lint-staged
npx husky init

.husky/pre-commit:
npx lint-staged

package.json:
{
  "lint-staged": {
    "*.js": ["eslint --fix", "prettier --write"]
  }
}

```

Recommended Minimal Pipeline for a Solo MERN Project

On push to any branch:

1. Install dependencies
2. Lint
3. Run unit tests
4. Run integration tests (against a test DB)

On merge to main:

5. (Above steps, then)
6. Auto-deploy backend (Render) + frontend (Vercel)

6. DevOps Basics

6.1 Overview of DevOps Culture and Practices

DevOps is the philosophy of breaking down the wall between “writing code” (Dev) and “running code in production” (Ops). Core practices:

- **Automation** — automate builds, tests, deployments, and infrastructure provisioning.
- **Continuous Monitoring** — know when something breaks before your users tell you.
- **Infrastructure as Code (IaC)** — define your servers/services in version-controlled config files, not manual clicks.
- **Collaboration and shared responsibility** — “you build it, you run it.” As a solo dev, this is automatically true — own it deliberately.
- **Fast feedback loops** — the faster you find out something’s broken, the cheaper it is to fix.

6.2 Infrastructure as Code, Monitoring, and Automation Basics

Infrastructure as Code (IaC) — Even Lightweight Versions Count

You don’t need Terraform/Kubernetes for a solo MERN app, but you should still **codify your environment**:

Dockerfile (backend):

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci --omit=dev
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["node", "src/server.js"]
```

docker-compose.yml (local dev with backend + frontend + (optional) local Mongo):

```
version: '3.8'
```

```
services:
```

```
  backend:
```

```
    build: ./backend
```

```
    ports:
```

```
      - "5000:5000"
```

```
    env_file: ./backend/.env
```

```
    depends_on:
```

```
      - mongo
```

```
  frontend:
```

```
    build: ./frontend
```

```
    ports:
```

```
      - "3000:3000"
```



```

mongo:
  image: mongo:7
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db

volumes:
  mongo_data:

```

Even if you deploy to Atlas + Render (not Docker), having a working `docker-compose.yml` means:

- Anyone (including future-you on a new machine) can run `docker compose up` and have a working dev environment in minutes.
- It documents your exact dependency versions and required services.

Environment Configuration

Maintain a clear separation between environments:

```

.env                # local dev (gitignored)
.env.example        # template, committed to git
.env.test           # used during automated tests

# .env.example
PORT=5000
MONGO_URI=mongodb://localhost:27017/feria_dev
JWT_SECRET=replace_me
NODE_ENV=development

```

Monitoring (Lightweight, Free Options)

Need	Tool
Uptime monitoring	UptimeRobot (free, pings your API every 5 min, emails you if it's down)
Error tracking	Sentry (free tier — catches unhandled exceptions in production with stack traces)
Application logs	Render/Railway built-in log viewers
Performance/APM	Render metrics dashboard (basic CPU/memory)

Sentry setup example:

```

const Sentry = require('@sentry/node');

Sentry.init({ dsn: process.env.SENTRY_DSN });

app.use(Sentry.Handlers.requestHandler());

```

```
// ... routes ...  
app.use(Sentry.Handlers.errorHandler());
```

Automation Basics

- **Automated backups** — MongoDB Atlas has built-in automated backups on most tiers; verify they're enabled.
- **Scheduled tasks** — use `node-cron` for in-app scheduled jobs (e.g., cleanup expired tokens), or platform-level cron jobs (Render Cron Jobs) for anything that should run independent of your main server process.
- **Dependency updates** — Dependabot or `npm outdated` checked periodically.

6.3 Managing Deployments, Scaling, and Environments Solo

Environments

At minimum, maintain two environments:

Environment	Purpose	Branch	Database
Development	Local coding	any feature branch	Local Mongo or Atlas dev cluster
Production	Live app	<code>main</code>	Atlas production cluster

A **staging** environment (mirroring production, deployed from a `develop` branch) is a nice-to-have once your app has real users, but isn't essential when solo and early-stage.

Deployment Strategy for a Solo Dev

1. Work on feature branches (`feature/fix-slug-index`).
2. Open a PR to `main` (even solo — this triggers CI and gives you a final review checkpoint).
3. CI runs lint + tests automatically.
4. Merge → auto-deploy via Render/Vercel.
5. Manually smoke-test the live app (visit key pages, hit key API endpoints).

Rollbacks

Know how to roll back *before* you need to:

- Render/Railway: redeploy a previous successful build from the dashboard (usually one click).
- Git: `git revert <commit>` and push — triggers a new deploy with the old code.

Scaling Basics (For When You Need It)

- **Vertical scaling** — bump your hosting plan's CPU/RAM (simplest first step).
- **Horizontal scaling** — run multiple instances behind a load balancer (most platforms support this with a slider/config change).
- **Database scaling** — MongoDB Atlas supports read replicas and sharding, but for most solo projects, proper **indexing** (see Section 2) solves 90% of performance issues before scaling infrastructure is needed.

- **Caching layer** — introduce Redis for session storage, rate-limiting counters, or caching expensive query results once you notice repeated, slow queries.

Managing Secrets

- Never commit secrets to git (`.env` in `.gitignore`).
- Store production secrets in your hosting platform's environment variable dashboard.
- Rotate secrets (especially `JWT_SECRET`, DB credentials) if ever accidentally exposed.

7. Unit Testing and QA Testing

7.1 Importance of Testing in Software Quality

Testing is how you get **confidence to change code** without breaking things — especially critical for a solo dev who has no second pair of eyes reviewing every change.

- **Unit tests** — test individual functions/modules in isolation (e.g., does `slugify("Hello World")` return `"hello-world"`?).
- **Integration tests** — test how multiple parts work together (e.g., does `POST /api/articles` actually create a document in the database and return 201?).
- **End-to-End (E2E) tests** — test the full user flow through the UI (e.g., Cypress/Playwright simulating a user creating an article in the browser).
- **QA Testing** — broader quality assurance: manual exploratory testing, checking edge cases, usability, cross-browser behavior.

The Testing Pyramid

```
      /\
     /E2E\      <- few, slow, expensive, high confidence
    /-----\
   /Integr. \   <- some, medium speed
  /-----\
 /   Unit   \   <- many, fast, cheap
/-----\
```

For a solo dev: invest most heavily in **unit tests** for business logic (services), a **moderate number of integration tests** for API endpoints, and a **handful of E2E tests** for critical user flows (signup, login, core CRUD).

7.2 Writing Effective Unit Tests for Backend Code

Setup (Jest)

```
npm install --save-dev jest
```

package.json:

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}
```

Example: Testing a Pure Function

```
// utils/slugify.js
function slugify(text) {
  return text
    .toLowerCase()
```

```

        .trim()
        .replace(/[^\w\s-]/g, '')
        .replace(/[\s_-]+/g, '-')
        .replace(/^+|-$/g, '');
    }

module.exports = slugify;

// utils/slugify.test.js
const slugify = require('./slugify');

describe('slugify', () => {
    it('converts spaces to hyphens', () => {
        expect(slugify('Hello World')).toBe('hello-world');
    });

    it('removes special characters', () => {
        expect(slugify('Hello, World!')).toBe('hello-world');
    });

    it('handles multiple spaces and trims', () => {
        expect(slugify(' Hello World ')).toBe('hello-world');
    });

    it('handles already-lowercase, hyphenated input', () => {
        expect(slugify('hello-world')).toBe('hello-world');
    });
});

```

Example: Testing a Service With Mocked Dependencies

```

// services/articleService.test.js
const articleService = require('./articleService');
const Article = require('../models/Article');
const AppError = require('../utils/AppError');

jest.mock('../models/Article');

describe('articleService.createArticle', () => {
    afterEach(() => jest.clearAllMocks());

    it('throws if slug already exists', async () => {
        Article.findOne.mockResolvedValue({ slug: 'hello-world' });

        await expect(
            articleService.createArticle({ title: 'Hello World' }, 'user123')
        ).rejects.toThrow(AppError);
    });
});

```

```

it('creates an article when slug is unique', async () => {
  Article.findOne.mockResolvedValue(null);
  Article.create.mockResolvedValue({ title: 'Hello World', slug: 'hello-world' });

  const result = await articleService.createArticle({ title: 'Hello World' }, 'user123');

  expect(Article.create).toHaveBeenCalledWith(
    expect.objectContaining({ slug: 'hello-world', author: 'user123' })
  );
  expect(result.slug).toBe('hello-world');
});
});

```

What Makes a Test “Good”

1. **Fast** — unit tests should run in milliseconds; no real database/network calls.
2. **Isolated** — one test’s failure shouldn’t depend on another test’s state.
3. **Repeatable** — same result every time (no reliance on current date/time, random values, or external services without mocking).
4. **Descriptive names** — `it('returns 409 when slug already exists')`, not `it('test1')`.
5. **Arrange-Act-Assert structure** — set up data, call the function, check the result.

7.3 Automated Testing and Integration Testing

Integration Testing with Supertest

```
npm install --save-dev supertest mongodb-memory-server
```

```

// tests/integration/articles.test.js
const request = require('supertest');
const mongoose = require('mongoose');
const { MongoMemoryServer } = require('mongodb-memory-server');
const app = require('../../src/app');
const Article = require('../../src/models/Article');

```

```
let mongoServer;
```

```

beforeAll(async () => {
  mongoServer = await MongoMemoryServer.create();
  await mongoose.connect(mongoServer.getUri());
});

```

```

afterAll(async () => {
  await mongoose.disconnect();
  await mongoServer.stop();
});

```

```
afterEach(async () => {
```

```

    await Article.deleteMany({});
  });

describe('POST /api/articles', () => {
  it('creates an article and returns 201', async () => {
    const res = await request(app)
      .post('/api/articles')
      .set('Authorization', `Bearer ${validTestToken}`)
      .send({ title: 'My First Post', body: 'Hello!' });

    expect(res.status).toBe(201);
    expect(res.body.data.slug).toBe('my-first-post');
  });

  it('returns 409 when creating a duplicate slug', async () => {
    await Article.create({ title: 'My First Post', slug: 'my-first-post', author: testUserId });

    const res = await request(app)
      .post('/api/articles')
      .set('Authorization', `Bearer ${validTestToken}`)
      .send({ title: 'My First Post', body: 'Duplicate' });

    expect(res.status).toBe(409);
    expect(res.body.success).toBe(false);
  });
});

```

Using `mongodb-memory-server` spins up an in-memory MongoDB instance for tests — no need for a separate test database, and tests run fully isolated and fast.

Running Tests in CI

Reference back to Section 5 — your `npm test` script (running both unit and integration tests) should run automatically in GitHub Actions on every push.

7.4 Strategies for Quality Assurance When Working Alone

1. **Write a manual QA checklist** for core flows, and run through it before every deploy to production:

Pre-Deploy QA Checklist

- [] Can register a new user
- [] Can log in with correct credentials
- [] Cannot log in with wrong password
- [] Can create an article with a unique title
- [] Creating an article with a duplicate title returns a clear error (not a 500)
- [] Can view article list (pagination works)
- [] Can edit own article
- [] Cannot edit another user's article

- [] Mobile layout doesn't break on key pages

2. **Test edge cases deliberately**, not just the “happy path”:

- Empty strings, missing fields, wrong data types
- Very long inputs
- Special characters / emoji in text fields (relevant to your slug generation!)
- Concurrent requests (e.g., two requests creating the same slug at the same time — this is exactly the kind of race condition that unique indexes protect against)

3. **Use linters and type checking** as a first line of defense:

- ESLint catches bugs before they’re even run.
- Consider migrating critical modules to TypeScript over time — type errors caught at compile time are bugs you’ll never ship.

4. **Code review yourself** — before merging a PR (even to yourself), use GitHub’s PR diff view to review your own changes with fresh eyes. You’ll catch console.logs, commented-out code, and typos you missed while writing.

5. **Track bugs as issues**, not just fix-and-forget. A bug label on your GitHub Issues board, with a short reproduction note, helps you spot patterns (e.g., “I keep having issues with unique index assumptions — maybe I need a pre-save validation hook everywhere I generate slugs”).

6. **“Bug bash” sessions** — periodically (e.g., once per sprint), spend 30 minutes deliberately trying to break your own app: weird inputs, rapid clicking, slow network simulation (Chrome DevTools throttling), etc.

7. **Staging environment smoke tests** — if you have a staging deploy, always test there before promoting to production, even for “small” changes.

Putting It All Together: A Solo Dev Weekly Rhythm

A sample weekly cadence that ties every section of this guide together:

Day	Activity
Monday	Sprint planning (15-30 min): review backlog, pick 3-7 tasks for the week, update Kanban board
Tue-Thu	Development: write code following layered MVC, commit small/focused changes, write unit tests alongside new logic
Daily	2-min “standup” journal entry: what you did, what’s next, any blockers
Before each PR	Run lint + tests locally; self-review the diff
On push	CI pipeline runs automatically (lint, unit tests, integration tests)
On merge to main	Auto-deploy to production via Render/Vercel
Friday	Manual QA checklist run-through on production; retrospective (10 min: what worked, what didn’t, one change for next week); update CHANGELOG.md

Final Notes

Being a “solo chud” dev team isn’t about cutting corners — it’s about choosing the **minimum viable process** that still gives you the benefits of good engineering practice: clear priorities (Agile), well-modeled data (Relational DB thinking), maintainable code (Backend + MVC), safety nets (CI/CD + Testing), and operational confidence (DevOps).

Start small. You don’t need all seven sections perfectly implemented on day one. A reasonable starting order for an existing project like feria-webprog:

1. Fix immediate bugs (your slug index issue) — and **write a test that would have caught it**.
2. Set up a basic Kanban board for visibility.
3. Add a GitHub Actions CI workflow (lint + test on push).
4. Audit your project structure against the layered MVC layout — refactor incrementally, not all at once.
5. Add a security checklist pass (helmet, rate limiting, env vars).
6. Build out your unit/integration test suite for core flows.
7. Set up basic monitoring (UptimeRobot + Sentry) once deployed.

Good luck — you’ve got this. One commit at a time.